DATABASE MANAGEMENT SYSTEMS · ASSIGNMENT 6

Implementation of Data Integrity Constraints

Primary keys, foreign keys, UNIQUE, CHECK and DEFAULT constraints – implementation and verification

Project	FoodExpress – Online Food Delivery System	Submitted by	Dhruvan Karthik
DBMS	MySQL Community Server 8.0.46	GitHub	github.com/hidhruvan-prog
Database	`foodexpress`	Date	25 October 2026

1. Objective

Integrity constraints are rules declared in the schema that the DBMS checks on every INSERT, UPDATE and DELETE, so that invalid data is rejected no matter which application or user tries to write it. This assignment implements the constraints of the FoodExpress database, explains the purpose of each one, and executes test statements that prove the rules and the relationships between tables are enforced.

2. Types of Integrity Used

Integrity type	Ensures that…	Implemented with
Entity integrity	every row is uniquely identifiable	PRIMARY KEY, NOT NULL
Key integrity	real-world identifiers are not duplicated	UNIQUE (single-column and composite)
Domain integrity	each value is of the right type and range	data types, ENUM, CHECK, NOT NULL
Referential integrity	references between tables always point to existing rows	FOREIGN KEY with ON UPDATE / ON DELETE actions, composite FK
User-defined (business) integrity	rules specific to food delivery hold	multi-column CHECK, generated column + UNIQUE, BEFORE INSERT triggers
Default values	sensible values are filled in when a column is omitted	DEFAULT

3. Primary Keys and Foreign Keys

Primary and foreign keys were declared when the tables were created in Assignment 5 (`sql/01_create_database.sql`). The data dictionary confirms them:

```
SELECT TABLE_NAME, CONSTRAINT_NAME, CONSTRAINT_TYPE
FROM information_schema.TABLE_CONSTRAINTS
WHERE TABLE_SCHEMA = 'foodexpress'
  AND CONSTRAINT_TYPE IN ('PRIMARY KEY', 'FOREIGN KEY')
ORDER BY CONSTRAINT_TYPE DESC, TABLE_NAME, CONSTRAINT_NAME;
```

mysql — foodexpress

```
mysql> SELECT TABLE_NAME, CONSTRAINT_NAME, CONSTRAINT_TYPE
    -> FROM information_schema.TABLE_CONSTRAINTS
    -> WHERE TABLE_SCHEMA = 'foodexpress'
    ->   AND CONSTRAINT_TYPE IN ('PRIMARY KEY', 'FOREIGN KEY')
    -> ORDER BY CONSTRAINT_TYPE DESC, TABLE_NAME, CONSTRAINT_NAME;
+-----------------+--------------------------+-----------------+
| TABLE_NAME      | CONSTRAINT_NAME          | CONSTRAINT_TYPE |
+-----------------+--------------------------+-----------------+
| addresses       | PRIMARY                  | PRIMARY KEY     |
| customers       | PRIMARY                  | PRIMARY KEY     |
| delivery_agents | PRIMARY                  | PRIMARY KEY     |
| menu_items      | PRIMARY                  | PRIMARY KEY     |
| order_items     | PRIMARY                  | PRIMARY KEY     |
| orders          | PRIMARY                  | PRIMARY KEY     |
| payments        | PRIMARY                  | PRIMARY KEY     |
| restaurants     | PRIMARY                  | PRIMARY KEY     |
| reviews         | PRIMARY                  | PRIMARY KEY     |
| addresses       | fk_addresses_customer    | FOREIGN KEY     |
| menu_items      | fk_menu_items_restaurant | FOREIGN KEY     |
| order_items     | fk_order_items_item      | FOREIGN KEY     |
| order_items     | fk_order_items_order     | FOREIGN KEY     |
| orders          | fk_orders_address_owner  | FOREIGN KEY     |
| orders          | fk_orders_agent          | FOREIGN KEY     |
| orders          | fk_orders_customer       | FOREIGN KEY     |
| orders          | fk_orders_restaurant     | FOREIGN KEY     |
| payments        | fk_payments_order        | FOREIGN KEY     |
| reviews         | fk_reviews_order         | FOREIGN KEY     |
+-----------------+--------------------------+-----------------+
19 rows in set (0.01 sec)
```

Table	Primary key	Purpose of the key
customers / restaurants / delivery_agents	customer_id / restaurant_id / agent_id	Surrogate AUTO_INCREMENT keys: stable even if a name, phone or e-mail changes.
addresses	address_id	Identifies one saved address; (address_id, customer_id) is also UNIQUE so the composite foreign key in orders can reference it.
menu_items	item_id	Identifies a dish; dish names are unique only within a restaurant.
orders	order_id	Order number shown to customers and used by every child table.
order_items	order_item_id	Identifies a line; (order_id, item_id) is additionally UNIQUE.
payments / reviews	payment_id / review_id	With UNIQUE (order_id) these make the 1 : 1 relationships with orders.

The ten foreign keys and their referential actions are listed in Assignment 5, §7.3. In short: **CASCADE** is used where the child is part of the parent (order → items, payment, review; customer → addresses); **RESTRICT** protects history (customers, restaurants, addresses and menu items that are used by orders cannot be deleted); and **SET NULL** lets an order survive if its rider record is removed.

4. Adding UNIQUE, CHECK and DEFAULT Constraints

The constraints are added with `ALTER TABLE` (`sql/02_constraints.sql`). Every constraint is named, so violations report a meaningful name. MySQL 8.0.16 and later enforce CHECK constraints (earlier versions parsed and silently ignored them). Defaults are set with `MODIFY`, because the column definition is restated when a default is added.

Constraints on customers

```sql
-- customers
ALTER TABLE customers
    MODIFY registration_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ADD CONSTRAINT uq_customers_email UNIQUE (email),
    ADD CONSTRAINT uq_customers_phone UNIQUE (phone),
    ADD CONSTRAINT chk_customers_email CHECK (email LIKE '%_@_%._%'),
    ADD CONSTRAINT chk_customers_phone CHECK (phone REGEXP '^[6-9][0-9]{9}$');
```

mysql — foodexpress

```
mysql> ALTER TABLE customers
    ->     MODIFY registration_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->     ADD CONSTRAINT uq_customers_email UNIQUE (email),
    ->     ADD CONSTRAINT uq_customers_phone UNIQUE (phone),
    ->     ADD CONSTRAINT chk_customers_email CHECK (email LIKE '%_@_%._%'),
    ->     ADD CONSTRAINT chk_customers_phone CHECK (phone REGEXP '^[6-9][0-9]{9}$');
Query OK, 0 rows affected (0.08 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Constraints on addresses

```sql
-- addresses
ALTER TABLE addresses
    MODIFY label      VARCHAR(20) NOT NULL DEFAULT 'Home',
    MODIFY is_default BOOLEAN     NOT NULL DEFAULT FALSE,
    MODIFY is_active  BOOLEAN     NOT NULL DEFAULT TRUE,
    ADD CONSTRAINT chk_addresses_label   CHECK (label IN ('Home','Work','Other')),
    ADD CONSTRAINT chk_addresses_pincode CHECK (pincode REGEXP '^[1-9][0-9]{5}$');
```

mysql — foodexpress

```
mysql> ALTER TABLE addresses
    ->     MODIFY label      VARCHAR(20) NOT NULL DEFAULT 'Home',
    ->     MODIFY is_default BOOLEAN     NOT NULL DEFAULT FALSE,
    ->     MODIFY is_active  BOOLEAN     NOT NULL DEFAULT TRUE,
    ->     ADD CONSTRAINT chk_addresses_label   CHECK (label IN ('Home','Work','Other')),
    ->     ADD CONSTRAINT chk_addresses_pincode CHECK (pincode REGEXP '^[1-9][0-9]{5}$');
Query OK, 0 rows affected (0.09 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Constraints on addresses

```sql
-- At most one default address per customer. MySQL has no partial unique
-- index, so a generated column holds customer_id only for the default row
-- (NULL otherwise) and a UNIQUE key on it allows any number of NULLs.
ALTER TABLE addresses
    ADD COLUMN default_owner INT
        GENERATED ALWAYS AS (IF(is_default, customer_id, NULL)) VIRTUAL,
    ADD CONSTRAINT uq_addresses_one_default UNIQUE (default_owner);
```

mysql — foodexpress

```
mysql> ALTER TABLE addresses
    ->     ADD COLUMN default_owner INT
    ->         GENERATED ALWAYS AS (IF(is_default, customer_id, NULL)) VIRTUAL,
    ->     ADD CONSTRAINT uq_addresses_one_default UNIQUE (default_owner);
Query OK, 0 rows affected (0.11 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Constraints on restaurants

```
-- restaurants
ALTER TABLE restaurants
    MODIFY is_active BOOLEAN NOT NULL DEFAULT TRUE,
    ADD CONSTRAINT uq_restaurants_phone   UNIQUE (contact_phone),
    ADD CONSTRAINT chk_restaurants_pincode CHECK (pincode REGEXP '^[1-9][0-9]{5}$'),
    ADD CONSTRAINT chk_restaurants_phone   CHECK (contact_phone REGEXP '^[0-9]{10,11}$');
```

mysql — foodexpress

```
mysql> ALTER TABLE restaurants
    ->     MODIFY is_active BOOLEAN NOT NULL DEFAULT TRUE,
    ->     ADD CONSTRAINT uq_restaurants_phone   UNIQUE (contact_phone),
    ->     ADD CONSTRAINT chk_restaurants_pincode CHECK (pincode REGEXP '^[1-9][0-9]{5}$'),
    ->     ADD CONSTRAINT chk_restaurants_phone   CHECK (contact_phone REGEXP '^[0-9]{10,11}$');
Query OK, 0 rows affected (0.11 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Constraints on menu_items

```
-- menu_items
ALTER TABLE menu_items
    MODIFY is_veg       BOOLEAN NOT NULL DEFAULT TRUE,
    MODIFY is_available BOOLEAN NOT NULL DEFAULT TRUE,
    ADD CONSTRAINT uq_menu_items_name     UNIQUE (restaurant_id, item_name),
    ADD CONSTRAINT chk_menu_items_price   CHECK (price > 0),
    ADD CONSTRAINT chk_menu_items_category CHECK (category IN
        ('Starter','Main Course','Bread','Rice','Dessert','Beverage','Snack'));
```

mysql — foodexpress

```
mysql> ALTER TABLE menu_items
    ->     MODIFY is_veg       BOOLEAN NOT NULL DEFAULT TRUE,
    ->     MODIFY is_available BOOLEAN NOT NULL DEFAULT TRUE,
    ->     ADD CONSTRAINT uq_menu_items_name     UNIQUE (restaurant_id, item_name),
    ->     ADD CONSTRAINT chk_menu_items_price   CHECK (price > 0),
    ->     ADD CONSTRAINT chk_menu_items_category CHECK (category IN
    ->         ('Starter','Main Course','Bread','Rice','Dessert','Beverage','Snack'));
Query OK, 0 rows affected (0.08 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Constraints on delivery_agents

```
-- delivery_agents
ALTER TABLE delivery_agents
    MODIFY is_available BOOLEAN NOT NULL DEFAULT TRUE,
    MODIFY joined_on    DATE    NOT NULL DEFAULT (CURRENT_DATE),
    ADD CONSTRAINT uq_agents_phone   UNIQUE (phone),
    ADD CONSTRAINT uq_agents_vehicle UNIQUE (vehicle_number),
    ADD CONSTRAINT chk_agents_phone  CHECK (phone REGEXP '^[6-9][0-9]{9}$'),
    ADD CONSTRAINT chk_agents_vehicle
        CHECK (vehicle_number REGEXP '^[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}$');
```

mysql — foodexpress

```
mysql> ALTER TABLE delivery_agents
    ->     MODIFY is_available BOOLEAN NOT NULL DEFAULT TRUE,
    ->     MODIFY joined_on    DATE    NOT NULL DEFAULT (CURRENT_DATE),
    ->     ADD CONSTRAINT uq_agents_phone   UNIQUE (phone),
    ->     ADD CONSTRAINT uq_agents_vehicle UNIQUE (vehicle_number),
    ->     ADD CONSTRAINT chk_agents_phone  CHECK (phone REGEXP '^[6-9][0-9]{9}$'),
    ->     ADD CONSTRAINT chk_agents_vehicle
    ->         CHECK (vehicle_number REGEXP '^[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}$');
Query OK, 0 rows affected (0.08 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Constraints on orders

```
-- orders
ALTER TABLE orders
    MODIFY order_date   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    MODIFY order_status ENUM('Placed','Preparing','Out for Delivery',
                             'Delivered','Cancelled') NOT NULL DEFAULT 'Placed',
    MODIFY delivery_fee DECIMAL(6,2) NOT NULL DEFAULT 40.00,
    ADD CONSTRAINT chk_orders_fee CHECK (delivery_fee BETWEEN 0 AND 200),
    -- delivered_at is filled exactly when the order is Delivered
    ADD CONSTRAINT chk_orders_delivered CHECK (
        (order_status =  'Delivered' AND delivered_at IS NOT NULL) OR
        (order_status <> 'Delivered' AND delivered_at IS NULL)),
    ADD CONSTRAINT chk_orders_delivery_time
        CHECK (delivered_at IS NULL OR delivered_at > order_date);
```

mysql — foodexpress

```
mysql> ALTER TABLE orders
    ->     MODIFY order_date   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->     MODIFY order_status ENUM('Placed','Preparing','Out for Delivery',
    ->                              'Delivered','Cancelled') NOT NULL DEFAULT 'Placed',
    ->     MODIFY delivery_fee DECIMAL(6,2) NOT NULL DEFAULT 40.00,
    ->     ADD CONSTRAINT chk_orders_fee CHECK (delivery_fee BETWEEN 0 AND 200),
    ->
    ->     ADD CONSTRAINT chk_orders_delivered CHECK (
    ->         (order_status =  'Delivered' AND delivered_at IS NOT NULL) OR
    ->         (order_status <> 'Delivered' AND delivered_at IS NULL)),
    ->     ADD CONSTRAINT chk_orders_delivery_time
    ->         CHECK (delivered_at IS NULL OR delivered_at > order_date);
Query OK, 0 rows affected (0.11 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Constraints on order_items

```
-- order_items
ALTER TABLE order_items
    ADD CONSTRAINT uq_order_items_line UNIQUE (order_id, item_id),
    ADD CONSTRAINT chk_order_items_qty   CHECK (quantity BETWEEN 1 AND 50),
    ADD CONSTRAINT chk_order_items_price CHECK (price_at_order > 0);
```

mysql — foodexpress

```
mysql> ALTER TABLE order_items
    ->     ADD CONSTRAINT uq_order_items_line UNIQUE (order_id, item_id),
    ->     ADD CONSTRAINT chk_order_items_qty   CHECK (quantity BETWEEN 1 AND 50),
    ->     ADD CONSTRAINT chk_order_items_price CHECK (price_at_order > 0);
Query OK, 0 rows affected (0.08 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Constraints on payments

```
-- payments
ALTER TABLE payments
    MODIFY payment_status ENUM('Pending','Completed','Failed','Refunded')
                          NOT NULL DEFAULT 'Pending',
    MODIFY payment_date   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ADD CONSTRAINT uq_payments_order  UNIQUE (order_id),
    ADD CONSTRAINT chk_payments_amount CHECK (amount >= 0);
```

```
mysql — foodexpress
mysql> ALTER TABLE payments
    ->     MODIFY payment_status ENUM('Pending','Completed','Failed','Refunded')
    ->                          NOT NULL DEFAULT 'Pending',
    ->     MODIFY payment_date   DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->     ADD CONSTRAINT uq_payments_order  UNIQUE (order_id),
    ->     ADD CONSTRAINT chk_payments_amount CHECK (amount >= 0);
Query OK, 0 rows affected (0.10 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

Constraints on reviews

```sql
-- reviews
ALTER TABLE reviews
    MODIFY review_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ADD CONSTRAINT uq_reviews_order UNIQUE (order_id),
    ADD CONSTRAINT chk_reviews_food  CHECK (food_rating BETWEEN 1 AND 5),
    ADD CONSTRAINT chk_reviews_agent
        CHECK (agent_rating IS NULL OR agent_rating BETWEEN 1 AND 5);
```

```
mysql — foodexpress
mysql> ALTER TABLE reviews
    ->     MODIFY review_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ->     ADD CONSTRAINT uq_reviews_order UNIQUE (order_id),
    ->     ADD CONSTRAINT chk_reviews_food  CHECK (food_rating BETWEEN 1 AND 5),
    ->     ADD CONSTRAINT chk_reviews_agent
    ->         CHECK (agent_rating IS NULL OR agent_rating BETWEEN 1 AND 5);
Query OK, 0 rows affected (0.09 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

4.1 Triggers for rules that span two tables

A CHECK constraint can only look at columns of the row being written, so the two rules below, which need data from another table, are implemented as `BEFORE INSERT` triggers that raise an error with `SIGNAL SQLSTATE '45000'`.

```
DELIMITER $$

-- a menu item can only be added to an order placed with its restaurant
CREATE TRIGGER trg_order_items_same_restaurant
BEFORE INSERT ON order_items
FOR EACH ROW
BEGIN
    IF (SELECT restaurant_id FROM menu_items WHERE item_id = NEW.item_id)
       <> (SELECT restaurant_id FROM orders WHERE order_id = NEW.order_id) THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Menu item does not belong to the order''s restaurant';
    END IF;
END$$

-- a review can only be written for an order that has been delivered
CREATE TRIGGER trg_reviews_only_delivered
BEFORE INSERT ON reviews
FOR EACH ROW
BEGIN
    IF (SELECT order_status FROM orders WHERE order_id = NEW.order_id)
       <> 'Delivered' THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Only delivered orders can be reviewed';
    END IF;
END$$

DELIMITER ;
```

```
SELECT TRIGGER_NAME, ACTION_TIMING, EVENT_MANIPULATION, EVENT_OBJECT_TABLE
FROM information_schema.TRIGGERS
WHERE TRIGGER_SCHEMA = 'foodexpress';
```

mysql — foodexpress

```
mysql> SELECT TRIGGER_NAME, ACTION_TIMING, EVENT_MANIPULATION, EVENT_OBJECT_TABLE
    -> FROM information_schema.TRIGGERS
    -> WHERE TRIGGER_SCHEMA = 'foodexpress';
+--------------------------------+--------------+--------------------+--------------------+
| TRIGGER_NAME                   | ACTION_TIMING | EVENT_MANIPULATION | EVENT_OBJECT_TABLE |
+--------------------------------+--------------+--------------------+--------------------+
| trg_order_items_same_restaurant | BEFORE        | INSERT             | order_items        |
| trg_reviews_only_delivered      | BEFORE        | INSERT             | reviews            |
+--------------------------------+--------------+--------------------+--------------------+
2 rows in set (0.00 sec)
```

5. Purpose of Each Constraint

Constraint	Table	Type	Purpose
uq_customers_email, uq_customers_phone	customers	UNIQUE	One account per e-mail / mobile number; both are used for login and OTPs.
chk_customers_email	customers	CHECK	E-mail must have the shape text@text.text.
chk_customers_phone, chk_agents_phone	customers, delivery_agents	CHECK	Indian mobile number: 10 digits starting with 6–9.
registration_date DEFAULT CURRENT_TIMESTAMP	customers	DEFAULT	Sign-up time is recorded automatically.
chk_addresses_label	addresses	CHECK	Label limited to Home, Work or Other (DEFAULT 'Home').
chk_addresses_pincode, chk_restaurants_pincode	addresses, restaurants	CHECK	Valid six-digit Indian PIN code, not starting with 0.

Constraint	Table	Type	Purpose
uq_addresses_one_default (+ generated default_owner)	addresses	UNIQUE	At most one default address per customer.
is_default FALSE, is_active TRUE	addresses	DEFAULT	New addresses are active and not default unless stated.
uq_restaurants_phone	restaurants	UNIQUE	Each outlet has its own contact number.
chk_restaurants_phone	restaurants	CHECK	10–11 digit phone (mobile or STD landline).
uq_menu_items_name	menu_items	UNIQUE	A restaurant cannot list the same dish twice.
chk_menu_items_price, chk_menu_items_category	menu_items	CHECK	Price must be positive; category from a fixed list used by the app's menu tabs.
is_veg TRUE, is_available TRUE	menu_items	DEFAULT	New dishes are orderable by default.
uq_agents_phone, uq_agents_vehicle	delivery_agents	UNIQUE	One rider per phone and per registered vehicle.
chk_agents_vehicle	delivery_agents	CHECK	Indian registration format, e.g. KA01AB1234.
joined_on DEFAULT (CURRENT_DATE)	delivery_agents	DEFAULT	Expression default: today's date.
order_date, order_status, delivery_fee	orders	DEFAULT	New orders start as 'Placed', timestamped now, with the standard ₹40 fee.
chk_orders_fee	orders	CHECK	Delivery fee between ₹0 and ₹200.
chk_orders_delivered	orders	CHECK	delivered_at is set if and only if the status is 'Delivered'.
chk_orders_delivery_time	orders	CHECK	Delivery cannot happen before the order was placed.
uq_order_items_line	order_items	UNIQUE	Each dish appears once per order; quantity holds the count.
chk_order_items_qty, chk_order_items_price	order_items	CHECK	Quantity 1–50; price snapshot must be positive.
uq_payments_order, uq_reviews_order	payments, reviews	UNIQUE	Turns the order → payment and order → review links into 1 : 1.
chk_payments_amount	payments	CHECK	Amount cannot be negative.
payment_status 'Pending', payment_date now	payments	DEFAULT	A new payment awaits confirmation.
chk_reviews_food, chk_reviews_agent	reviews	CHECK	Ratings are 1–5 stars; the rider rating is optional (NULL).
trg_order_items_same_restaurant	order_items	TRIGGER	A dish must come from the restaurant the order was placed with.
trg_reviews_only_delivered	reviews	TRIGGER	Only delivered orders can be reviewed.

5.1 Constraint inventory from the data dictionary

```sql
SELECT CONSTRAINT_TYPE, COUNT(*) AS how_many
FROM information_schema.TABLE_CONSTRAINTS
WHERE TABLE_SCHEMA = 'foodexpress'
GROUP BY CONSTRAINT_TYPE
ORDER BY how_many DESC;
```

mysql — foodexpress

```
mysql> SELECT CONSTRAINT_TYPE, COUNT(*) AS how_many
    -> FROM information_schema.TABLE_CONSTRAINTS
    -> WHERE TABLE_SCHEMA = 'foodexpress'
    -> GROUP BY CONSTRAINT_TYPE
    -> ORDER BY how_many DESC;
+-----------------+----------+
| CONSTRAINT_TYPE | how_many |
+-----------------+----------+
| CHECK           |       18 |
| UNIQUE          |       11 |
| FOREIGN KEY     |       10 |
| PRIMARY KEY     |        9 |
+-----------------+----------+
4 rows in set (0.00 sec)
```

MySQL stores each CHECK expression with character-set prefixes (`_utf8mb4`) and backticks; the query removes them so the clauses are easier to read.

```sql
SELECT CONSTRAINT_NAME,
       REPLACE(REPLACE(CHECK_CLAUSE, '_utf8mb4', ''), '`', '') AS check_clause
FROM information_schema.CHECK_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA = 'foodexpress'
ORDER BY CONSTRAINT_NAME;
```

mysql — foodexpress

```
mysql> SELECT CONSTRAINT_NAME,
    ->        REPLACE(REPLACE(CHECK_CLAUSE, '_utf8mb4', ''), '`', '') AS check_clause
    -> FROM information_schema.CHECK_CONSTRAINTS
    -> WHERE CONSTRAINT_SCHEMA = 'foodexpress'
    -> ORDER BY CONSTRAINT_NAME;
+-------------------------+-----------------------------
```

mysql — foodexpress

```
mysql> SELECT TABLE_NAME, COLUMN_NAME, COLUMN_DEFAULT, EXTRA
    -> FROM information_schema.COLUMNS
    -> WHERE TABLE_SCHEMA = 'foodexpress'
    ->   AND COLUMN_DEFAULT IS NOT NULL
    -> ORDER BY TABLE_NAME, ORDINAL_POSITION;
+----------------------+-------------------+-------------------+-------------------+
| TABLE_NAME           | COLUMN_NAME       | COLUMN_DEFAULT    | EXTRA             |
+----------------------+-------------------+-------------------+-------------------+
| addresses            | label             | Home              |                   |
| addresses            | is_default        | 0                 |                   |
| addresses            | is_active         | 1                 |                   |
| customers            | registration_date | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| delivery_agents      | is_available      | 1                 |                   |
| delivery_agents      | joined_on         | curdate()         | DEFAULT_GENERATED |
| menu_items           | is_veg            | 1                 |                   |
| menu_items           | is_available      | 1                 |                   |
| orders               | order_date        | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| orders               | order_status      | Placed            |                   |
| orders               | delivery_fee      | 40.00             |                   |
| payments             | payment_status    | Pending           |                   |
| payments             | payment_date      | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| restaurants          | is_active         | 1                 |                   |
| reviews              | review_date       | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
| v_order_totals       | order_id          | 0                 |                   |
| v_order_totals       | delivery_fee      | 40.00             |                   |
| v_restaurant_ratings | restaurant_id     | 0                 |                   |
| v_restaurant_ratings | review_count      | 0                 |                   |
+----------------------+-------------------+-------------------+-------------------+
19 rows in set (0.01 sec)
```

6. Verifying Data Integrity

The tests below run on the empty database before the sample data of Assignment 7 is loaded. A small set of valid rows is inserted first (which also shows the DEFAULT values being applied), then each test tries one valid or invalid operation. Rejected statements return a MySQL error and change nothing. Tests that delete data run inside a transaction and end with `ROLLBACK`. At the end the test rows are removed.

6.1 Valid rows and DEFAULT values

Parent rows: two customers (registration_date omitted → DEFAULT)

```
INSERT INTO customers (customer_id, full_name, email, phone, password_hash) VALUES
    (1, 'Test Customer',   'test.customer@example.com', '9876543210', SHA2('demo-1', 256)),
    (2, 'Second Customer', 'second.customer@example.com', '9123456780', SHA2('demo-2', 256));
SELECT customer_id, full_name, registration_date FROM customers;
```

mysql — foodexpress

```
mysql> INSERT INTO customers (customer_id, full_name, email, phone, password_hash) VALUES
    ->     (1, 'Test Customer',   'test.customer@example.com', '9876543210', SHA2('demo-1', 256)),
    ->     (2, 'Second Customer', 'second.customer@example.com', '9123456780', SHA2('demo-2', 256));
Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> SELECT customer_id, full_name, registration_date FROM customers;
+-------------+----------------+---------------------+
| customer_id | full_name      | registration_date   |
+-------------+----------------+---------------------+
|           1 | Test Customer  | 2026-10-25 13:42:31 |
|           2 | Second Customer | 2026-10-25 13:42:31 |
+-------------+----------------+---------------------+
2 rows in set (0.00 sec)
```

Addresses (label and is_active omitted → DEFAULT 'Home' / TRUE)

```
INSERT INTO addresses (address_id, customer_id, address_line, city, state, pincode, is_default) VALUES
    (1, 1, '12, MG Road',     'Bengaluru', 'Karnataka',   '560001', TRUE),
    (2, 2, '5, Marine Drive', 'Mumbai',    'Maharashtra', '400020', TRUE);
SELECT address_id, customer_id, label, pincode, is_default, is_active FROM addresses;
```

mysql — foodexpress

```
mysql> INSERT INTO addresses (address_id, customer_id, address_line, city, state, pincode, is_default) VALUES
    ->     (1, 1, '12, MG Road',     'Bengaluru', 'Karnataka',   '560001', TRUE),
    ->     (2, 2, '5, Marine Drive', 'Mumbai',    'Maharashtra', '400020', TRUE);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> SELECT address_id, customer_id, label, pincode, is_default, is_active FROM addresses;
+------------+-------------+-------+---------+------------+-----------+
| address_id | customer_id | label | pincode | is_default | is_active |
+------------+-------------+-------+---------+------------+-----------+
|          1 |           1 | Home  | 560001  |          1 |         1 |
|          2 |           2 | Home  | 400020  |          1 |         1 |
+------------+-------------+-------+---------+------------+-----------+
2 rows in set (0.00 sec)
```

Restaurants, menu items and a rider (flags and joined_on → DEFAULT)

```
INSERT INTO restaurants (restaurant_id, restaurant_name, cuisine_type, address_line, city, pincode, contact_phone) VALUES
    (1, 'Test Kitchen',  'North Indian', '1, Brigade Road',  'Bengaluru', '560025', '08012345678'),
    (2, 'Other Kitchen', 'Chinese',      '2, Church Street', 'Bengaluru', '560001', '08087654321');
INSERT INTO menu_items (item_id, restaurant_id, item_name, price, category) VALUES
    (1, 1, 'Paneer Tikka', 220.00, 'Starter'),
    (2, 2, 'Veg Noodles',  180.00, 'Main Course');
INSERT INTO delivery_agents (agent_id, agent_name, phone, vehicle_number, city) VALUES
    (1, 'Test Rider', '9988776655', 'KA01AB1234', 'Bengaluru');
SELECT item_id, item_name, is_veg, is_available FROM menu_items;
SELECT agent_id, agent_name, is_available, joined_on FROM delivery_agents;
```

mysql — foodexpress

```
mysql> INSERT INTO restaurants (restaurant_id, restaurant_name, cuisine_type, address_line, city, pincode, con
tact_phone) VALUES
    ->     (1, 'Test Kitchen',  'North Indian', '1, Brigade Road',  'Bengaluru', '560025', '08012345678'),
    ->     (2, 'Other Kitchen', 'Chinese',      '2, Church Street', 'Bengaluru', '560001', '08087654321');
Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO menu_items (item_id, restaurant_id, item_name, price, category) VALUES
    ->     (1, 1, 'Paneer Tikka', 220.00, 'Starter'),
    ->     (2, 2, 'Veg Noodles',  180.00, 'Main Course');
Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO delivery_agents (agent_id, agent_name, phone, vehicle_number, city) VALUES
    ->     (1, 'Test Rider', '9988776655', 'KA01AB1234', 'Bengaluru');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT item_id, item_name, is_veg, is_available FROM menu_items;
+---------+--------------+--------+--------------+
| item_id | item_name    | is_veg | is_available |
+---------+--------------+--------+--------------+
|       1 | Paneer Tikka |      1 |            1 |
|       2 | Veg Noodles  |      1 |            1 |
+---------+--------------+--------+--------------+
2 rows in set (0.00 sec)

mysql> SELECT agent_id, agent_name, is_available, joined_on FROM delivery_agents;
+----------+------------+--------------+------------+
| agent_id | agent_name | is_available | joined_on  |
+----------+------------+--------------+------------+
|        1 | Test Rider |            1 | 2026-10-25 |
+----------+------------+--------------+------------+
1 row in set (0.00 sec)
```

An order, its line item and payment (order_date, order_status, delivery_fee, payment_status → DEFAULT)

```
INSERT INTO orders (order_id, customer_id, restaurant_id, address_id, agent_id)
VALUES (1, 1, 1, 1, 1);
INSERT INTO order_items (order_item_id, order_id, item_id, quantity, price_at_order)
VALUES (1, 1, 1, 2, 220.00);
INSERT INTO payments (payment_id, order_id, amount, payment_method)
VALUES (1, 1, 480.00, 'UPI');
SELECT order_id, order_date, order_status, delivery_fee, delivered_at FROM orders;
SELECT payment_id, order_id, amount, payment_status, payment_date FROM payments;
```

mysql — foodexpress

```
mysql> INSERT INTO orders (order_id, customer_id, restaurant_id, address_id, agent_id)
    -> VALUES (1, 1, 1, 1, 1);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO order_items (order_item_id, order_id, item_id, quantity, price_at_order)
    -> VALUES (1, 1, 1, 2, 220.00);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO payments (payment_id, order_id, amount, payment_method)
    -> VALUES (1, 1, 480.00, 'UPI');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT order_id, order_date, order_status, delivery_fee, delivered_at FROM orders;
+----------+---------------------+--------------+--------------+--------------+
| order_id | order_date          | order_status | delivery_fee | delivered_at |
+----------+---------------------+--------------+--------------+--------------+
|        1 | 2026-10-25 13:42:31 | Placed       |        40.00 | NULL         |
+----------+---------------------+--------------+--------------+--------------+
1 row in set (0.00 sec)

mysql> SELECT payment_id, order_id, amount, payment_status, payment_date FROM payments;
+------------+----------+--------+----------------+---------------------+
| payment_id | order_id | amount | payment_status | payment_date        |
+------------+----------+--------+----------------+---------------------+
|          1 |        1 | 480.00 | Pending        | 2026-10-25 13:42:31 |
+------------+----------+--------+----------------+---------------------+
1 row in set (0.00 sec)
```

6.2 Constraint tests

T1 PRIMARY KEY rejects a duplicate customer_id

mysql — foodexpress

```
mysql> INSERT INTO customers (customer_id, full_name, email, phone, password_hash)
    -> VALUES (1, 'Duplicate Id', 'dup@example.com', '9876500000', 'x');
ERROR 1062 (23000): Duplicate entry '1' for key 'customers.PRIMARY'
```

T2 NOT NULL rejects a menu item without a price

mysql — foodexpress

```
mysql> INSERT INTO menu_items (restaurant_id, item_name, category)
    -> VALUES (1, 'Mystery Dish', 'Starter');
ERROR 1364 (HY000): Field 'price' doesn't have a default value
```

T3 UNIQUE rejects a second account with the same e-mail

mysql — foodexpress

```
mysql> INSERT INTO customers (full_name, email, phone, password_hash)
    -> VALUES ('Copy Cat', 'test.customer@example.com', '9876500001', 'x');
ERROR 1062 (23000): Duplicate entry 'test.customer@example.com' for key 'customers.uq_customers_email'
```

T4 UNIQUE rejects a second rider with the same vehicle number

mysql — foodexpress

```
mysql> INSERT INTO delivery_agents (agent_name, phone, vehicle_number, city)
    -> VALUES ('Rider Two', '9988776600', 'KA01AB1234', 'Bengaluru');
ERROR 1062 (23000): Duplicate entry 'KA01AB1234' for key 'delivery_agents.uq_agents_vehicle'
```

T5 UNIQUE (restaurant_id, item_name) rejects a duplicate dish on one menu

mysql — foodexpress

```
mysql> INSERT INTO menu_items (restaurant_id, item_name, price, category)
    -> VALUES (1, 'Paneer Tikka', 250.00, 'Starter');
ERROR 1062 (23000): Duplicate entry '1-Paneer Tikka' for key 'menu_items.uq_menu_items_name'
```

T6 The generated-column UNIQUE key allows only one default address per customer

mysql — foodexpress

```
mysql> INSERT INTO addresses (customer_id, label, address_line, city, state, pincode, is_default)
    -> VALUES (1, 'Work', 'Manyata Tech Park', 'Bengaluru', 'Karnataka', '560045', TRUE);
ERROR 1062 (23000): Duplicate entry '1' for key 'addresses.uq_addresses_one_default'
```

T7 ...while a second, non-default address is accepted

mysql — foodexpress

```
mysql> INSERT INTO addresses (address_id, customer_id, label, address_line, city, state, pincode)
    -> VALUES (3, 1, 'Work', 'Manyata Tech Park', 'Bengaluru', 'Karnataka', '560045');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT address_id, customer_id, label, is_default, default_owner FROM addresses;
+------------+-------------+-------+------------+---------------+
| address_id | customer_id | label | is_default | default_owner |
+------------+-------------+-------+------------+---------------+
|          1 |           1 | Home  |          1 |             1 |
|          2 |           2 | Home  |          1 |             2 |
|          3 |           1 | Work  |          0 |          NULL |
+------------+-------------+-------+------------+---------------+
3 rows in set (0.00 sec)
```

T8 UNIQUE (order_id) on payments rejects a second payment for the same order

mysql — foodexpress

```
mysql> INSERT INTO payments (order_id, amount, payment_method) VALUES (1, 480.00, 'Card');
ERROR 1062 (23000): Duplicate entry '1' for key 'payments.uq_payments_order'
```

T9 UNIQUE (order_id, item_id) rejects the same dish twice in one order

mysql — foodexpress

```
mysql> INSERT INTO order_items (order_id, item_id, quantity, price_at_order) VALUES (1, 1, 1, 220.00);
ERROR 1062 (23000): Duplicate entry '1-1' for key 'order_items.uq_order_items_line'
```

T10 CHECK rejects an invalid mobile number

mysql — foodexpress

```
mysql> INSERT INTO customers (full_name, email, phone, password_hash)
    -> VALUES ('Bad Phone', 'bad.phone@example.com', '12345', 'x');
ERROR 3819 (HY000): Check constraint 'chk_customers_phone' is violated.
```

T11 CHECK rejects a malformed e-mail address

mysql — foodexpress

```
mysql> INSERT INTO customers (full_name, email, phone, password_hash)
    -> VALUES ('Bad Mail', 'bad-mail.example.com', '9876500002', 'x');
ERROR 3819 (HY000): Check constraint 'chk_customers_email' is violated.
```

T12 CHECK rejects a 5-digit pincode

mysql — foodexpress

```
mysql> INSERT INTO addresses (customer_id, address_line, city, state, pincode)
    -> VALUES (2, 'Andheri', 'Mumbai', 'Maharashtra', '40005');
ERROR 3819 (HY000): Check constraint 'chk_addresses_pincode' is violated.
```

T13 CHECK rejects a negative price

mysql — foodexpress

```
mysql> UPDATE menu_items SET price = -50 WHERE item_id = 1;
ERROR 3819 (HY000): Check constraint 'chk_menu_items_price' is violated.
```

T14 CHECK rejects a category outside the allowed list

mysql — foodexpress

```
mysql> INSERT INTO menu_items (restaurant_id, item_name, price, category)
    -> VALUES (1, 'Tomato Soup', 120, 'Soup');
ERROR 3819 (HY000): Check constraint 'chk_menu_items_category' is violated.
```

T15 CHECK rejects a quantity of zero

mysql — foodexpress

```
mysql> UPDATE order_items SET quantity = 0 WHERE order_item_id = 1;
ERROR 3819 (HY000): Check constraint 'chk_order_items_qty' is violated.
```

T16 ENUM rejects an unknown order status (strict mode)

mysql — foodexpress

```
mysql> UPDATE orders SET order_status = 'Shipped' WHERE order_id = 1;
ERROR 1265 (01000): Data truncated for column 'order_status' at row 1
```

T17 CHECK rejects a rider vehicle number in the wrong format

mysql — foodexpress

```
mysql> INSERT INTO delivery_agents (agent_name, phone, vehicle_number, city)
    -> VALUES ('Rider Three', '9988776611', 'BIKE-42', 'Bengaluru');
ERROR 3819 (HY000): Check constraint 'chk_agents_vehicle' is violated.
```

T18 CHECK: an order cannot be marked Delivered without delivered_at

mysql — foodexpress

```
mysql> UPDATE orders SET order_status = 'Delivered' WHERE order_id = 1;
ERROR 3819 (HY000): Check constraint 'chk_orders_delivered' is violated.
```

T19 CHECK: delivered_at cannot be earlier than order_date

mysql — foodexpress

```
mysql> UPDATE orders SET order_status = 'Delivered',
    ->                   delivered_at = order_date - INTERVAL 10 MINUTE
    -> WHERE order_id = 1;
ERROR 3819 (HY000): Check constraint 'chk_orders_delivery_time' is violated.
```

T20 A consistent update is accepted

mysql — foodexpress

```
mysql> UPDATE orders SET order_status = 'Delivered',
    ->                   delivered_at = order_date + INTERVAL 32 MINUTE
    -> WHERE order_id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT order_id, order_status, order_date, delivered_at FROM orders;
+----------+--------------+---------------------+---------------------+
| order_id | order_status | order_date          | delivered_at        |
+----------+--------------+---------------------+---------------------+
|        1 | Delivered    | 2026-10-25 13:42:31 | 2026-10-25 14:14:31 |
+----------+--------------+---------------------+---------------------+
1 row in set (0.00 sec)
```

T21 Trigger: a dish from another restaurant cannot be added to the order

mysql — foodexpress

```
mysql> INSERT INTO order_items (order_id, item_id, quantity, price_at_order)
    -> VALUES (1, 2, 1, 180.00);
ERROR 1644 (45000): Menu item does not belong to the order's restaurant
```

T22 Trigger: an order that is not delivered cannot be reviewed

mysql — foodexpress

```
mysql> INSERT INTO orders (order_id, customer_id, restaurant_id, address_id)
    -> VALUES (2, 2, 2, 2);
Query OK, 1 row affected (0.01 sec)

mysql> INSERT INTO reviews (order_id, food_rating) VALUES (2, 4);
ERROR 1644 (45000): Only delivered orders can be reviewed
```

T23 CHECK rejects a rating outside 1–5

mysql — foodexpress

```
mysql> INSERT INTO reviews (order_id, food_rating, agent_rating) VALUES (1, 6, 5);
ERROR 3819 (HY000): Check constraint 'chk_reviews_food' is violated.
```

T24 A valid review of the delivered order is accepted (review_date → DEFAULT)

mysql — foodexpress

```
mysql> INSERT INTO reviews (order_id, food_rating, agent_rating, comment)
    -> VALUES (1, 5, 4, 'Hot and fresh');
Query OK, 1 row affected (0.00 sec)

mysql> SELECT review_id, order_id, food_rating, agent_rating, review_date FROM reviews;
+-----------+----------+-------------+--------------+---------------------+
| review_id | order_id | food_rating | agent_rating | review_date         |
+-----------+----------+-------------+--------------+---------------------+
|         1 |        1 |           5 |            4 | 2026-10-25 13:42:33 |
+-----------+----------+-------------+--------------+---------------------+
1 row in set (0.00 sec)
```

T25 FOREIGN KEY rejects an order for a restaurant that does not exist

mysql — foodexpress

```
mysql> INSERT INTO orders (customer_id, restaurant_id, address_id) VALUES (1, 99, 1);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`foodexpress`.`orders`,
CONSTRAINT `fk_orders_restaurant` FOREIGN KEY (`restaurant_id`) REFERENCES `restaurants` (`restaurant_id`) ON
DELETE RESTRICT ON UPDATE CASCADE)
```

T26 Composite FOREIGN KEY rejects delivering customer 1's order to customer 2's address

mysql — foodexpress

```
mysql> INSERT INTO orders (customer_id, restaurant_id, address_id) VALUES (1, 1, 2);
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails (`foodexpress`.`orders`,
CONSTRAINT `fk_orders_address_owner` FOREIGN KEY (`address_id`, `customer_id`) REFERENCES `addresses` (`addres
s_id`, `customer_id`) ON DELETE RESTRICT ON UPDATE CASCADE)
```

T27 ON DELETE RESTRICT: a restaurant with menu items and orders cannot be deleted

mysql — foodexpress

```
mysql> DELETE FROM restaurants WHERE restaurant_id = 1;
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`foodexpress`.`menu_
items`, CONSTRAINT `fk_menu_items_restaurant` FOREIGN KEY (`restaurant_id`) REFERENCES `restaurants` (`restaur
ant_id`) ON DELETE RESTRICT ON UPDATE CASCADE)
```

T28 ON DELETE RESTRICT: a customer with order history cannot be deleted

mysql — foodexpress

```
mysql> DELETE FROM customers WHERE customer_id = 1;
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails (`foodexpress`.`order
s`, CONSTRAINT `fk_orders_address_owner` FOREIGN KEY (`address_id`, `customer_id`) REFERENCES `addresses` (`ad
dress_id`, `customer_id`) ON DELETE RESTRICT ON UPDATE CASCADE)
```

T29 ON DELETE SET NULL: removing a rider keeps the order but clears agent_id (rolled back)

mysql — foodexpress

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> DELETE FROM delivery_agents WHERE agent_id = 1;
Query OK, 1 row affected (0.00 sec)

mysql> SELECT order_id, agent_id, order_status FROM orders;
+----------+----------+--------------+
| order_id | agent_id | order_status |
+----------+----------+--------------+
|        1 |     NULL | Delivered    |
|        2 |     NULL | Placed       |
+----------+----------+--------------+
2 rows in set (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)
```

T30 ON DELETE CASCADE: deleting an order removes its items, payment and review (rolled back)

mysql — foodexpress

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> DELETE FROM orders WHERE order_id = 1;
Query OK, 1 row affected (0.00 sec)

mysql> SELECT (SELECT COUNT(*) FROM order_items WHERE order_id = 1) AS items_left,
    ->        (SELECT COUNT(*) FROM payments    WHERE order_id = 1) AS payments_left,
    ->        (SELECT COUNT(*) FROM reviews     WHERE order_id = 1) AS reviews_left;
+------------+--------------+--------------+
| items_left | payments_left | reviews_left |
+------------+--------------+--------------+
|          0 |             0 |            0 |
+------------+--------------+--------------+
1 row in set (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)
```

T31 ON DELETE CASCADE: deleting a customer without orders also deletes their addresses (rolled back)

mysql — foodexpress

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> DELETE FROM orders WHERE order_id = 2;
Query OK, 1 row affected (0.00 sec)

mysql> DELETE FROM customers WHERE customer_id = 2;
Query OK, 1 row affected (0.00 sec)

mysql> SELECT address_id, customer_id FROM addresses;
+------------+-------------+
| address_id | customer_id |
+------------+-------------+
|          1 |           1 |
|          3 |           1 |
+------------+-------------+
2 rows in set (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)
```

6.3 Removing the test rows

```sql
DELETE FROM reviews;      DELETE FROM payments;   DELETE FROM order_items;
DELETE FROM orders;       DELETE FROM menu_items; DELETE FROM delivery_agents;
DELETE FROM restaurants;  DELETE FROM addresses;  DELETE FROM customers;
SELECT (SELECT COUNT(*) FROM customers) AS customers, (SELECT COUNT(*) FROM orders) AS orders,
       (SELECT COUNT(*) FROM restaurants) AS restaurants;
```

mysql — foodexpress

```
mysql> DELETE FROM reviews;
Query OK, 1 row affected (0.01 sec)

mysql> DELETE FROM payments;
Query OK, 1 row affected (0.00 sec)

mysql> DELETE FROM order_items;
Query OK, 1 row affected (0.00 sec)

mysql> DELETE FROM orders;
Query OK, 2 rows affected (0.01 sec)

mysql> DELETE FROM menu_items;
Query OK, 2 rows affected (0.00 sec)

mysql> DELETE FROM delivery_agents;
Query OK, 1 row affected (0.00 sec)

mysql> DELETE FROM restaurants;
Query OK, 2 rows affected (0.00 sec)

mysql> DELETE FROM addresses;
Query OK, 3 rows affected (0.00 sec)

mysql> DELETE FROM customers;
Query OK, 2 rows affected (0.01 sec)

mysql> SELECT (SELECT COUNT(*) FROM customers) AS customers, (SELECT COUNT(*) FROM orders) AS orders,
    ->        (SELECT COUNT(*) FROM restaurants) AS restaurants;
+-----------+--------+-------------+
| customers | orders | restaurants |
+-----------+--------+-------------+
|         0 |      0 |           0 |
+-----------+--------+-------------+
1 row in set (0.00 sec)
```

7. Test Summary

Test	Integrity	Rule	Expected	Result
T1	Entity	PRIMARY KEY rejects a duplicate customer_id	Reject	Rejected – ERROR 1062
T2	Entity	NOT NULL rejects a menu item without a price	Reject	Rejected – ERROR 1364
T3	Key	UNIQUE rejects a second account with the same e-mail	Reject	Rejected – ERROR 1062
T4	Key	UNIQUE rejects a second rider with the same vehicle number	Reject	Rejected – ERROR 1062
T5	Key	UNIQUE (restaurant_id, item_name) rejects a duplicate dish on one menu	Reject	Rejected – ERROR 1062
T6	Key	The generated-column UNIQUE key allows only one default address per customer	Reject	Rejected – ERROR 1062
T7	Key	…while a second, non-default address is accepted	Accept	Accepted
T8	Key	UNIQUE (order_id) on payments rejects a second payment for the same order	Reject	Rejected – ERROR 1062
T9	Key	UNIQUE (order_id, item_id) rejects the same dish twice in one order	Reject	Rejected – ERROR 1062
T10	Domain	CHECK rejects an invalid mobile number	Reject	Rejected – ERROR 3819
T11	Domain	CHECK rejects a malformed e-mail address	Reject	Rejected – ERROR 3819
T12	Domain	CHECK rejects a 5-digit pincode	Reject	Rejected – ERROR 3819
T13	Domain	CHECK rejects a negative price	Reject	Rejected – ERROR 3819
T14	Domain	CHECK rejects a category outside the allowed list	Reject	Rejected – ERROR 3819
T15	Domain	CHECK rejects a quantity of zero	Reject	Rejected – ERROR 3819

Test	Integrity	Rule	Expected	Result
T16	Domain	ENUM rejects an unknown order status (strict mode)	Reject	**Rejected** – ERROR 1265
T17	Domain	CHECK rejects a rider vehicle number in the wrong format	Reject	**Rejected** – ERROR 3819
T18	Business rule	CHECK: an order cannot be marked Delivered without delivered_at	Reject	**Rejected** – ERROR 3819
T19	Business rule	CHECK: delivered_at cannot be earlier than order_date	Reject	**Rejected** – ERROR 3819
T20	Business rule	A consistent update is accepted	Accept	**Accepted**
T21	Business rule	Trigger: a dish from another restaurant cannot be added to the order	Reject	**Rejected** – ERROR 1644
T22	Business rule	Trigger: an order that is not delivered cannot be reviewed	Reject	**Rejected** – ERROR 1644
T23	Domain	CHECK rejects a rating outside 1–5	Reject	**Rejected** – ERROR 3819
T24	Business rule	A valid review of the delivered order is accepted (review_date → DEFAULT)	Accept	**Accepted**
T25	Referential	FOREIGN KEY rejects an order for a restaurant that does not exist	Reject	**Rejected** – ERROR 1452
T26	Referential	Composite FOREIGN KEY rejects delivering customer 1's order to customer 2's address	Reject	**Rejected** – ERROR 1452
T27	Referential	ON DELETE RESTRICT: a restaurant with menu items and orders cannot be deleted	Reject	**Rejected** – ERROR 1451
T28	Referential	ON DELETE RESTRICT: a customer with order history cannot be deleted	Reject	**Rejected** – ERROR 1451
T29	Referential	ON DELETE SET NULL: removing a rider keeps the order but clears agent_id (rolled back)	Accept	**Accepted**
T30	Referential	ON DELETE CASCADE: deleting an order removes its items, payment and review (rolled back)	Accept	**Accepted**
T31	Referential	ON DELETE CASCADE: deleting a customer without orders also deletes their addresses (rolled back)	Accept	**Accepted**

> All 31 tests behaved as expected: every invalid operation was rejected by the DBMS with an error identifying the violated rule, and every valid operation was accepted. Error 1062 = duplicate key (PRIMARY/UNIQUE), 1364 = missing NOT NULL value, 3819 = CHECK violated, 1265 = value not in ENUM, 1452 = child row has no parent (FK), 1451 = parent row still referenced (RESTRICT), 1644 = error raised by a trigger.

8. Conclusion

The FoodExpress database now enforces entity, key, domain, referential and business-rule integrity itself instead of trusting each application to validate data. Primary and foreign keys keep every relationship valid, UNIQUE constraints prevent duplicate identities, CHECK constraints keep values within their real-world ranges, DEFAULT values fill in timestamps, statuses and flags, and two triggers cover rules that involve two tables. The tests show each rule in action, so the sample data loaded in Assignment 7 must satisfy all of them.